

# Autoresearch Loops: A Causal Study of the Instruction File

Christian Block

Research project with Prof. Ramesh

September 20, 2026

## Abstract

An autoresearch loop is one in which a language model proposes, executes and interprets experiments without a human in the loop. In the form popularized by Karpathy’s autoresearch, that procedure is written in English in a single instruction file, `program.md`. This thesis treats the instruction file as a configuration vector. Five components of `program.md` are each written at two levels. All  $2^5 = 32$  combinations are generated and executed on two classification tasks with twenty replicates each, giving 1,280 runs. Four behavioral metrics read from the structured transcripts are then regressed on an orthogonal  $\pm 1$  coding of the components with a dataset fixed effect. Because the configurations are constructed instead of observed, the coefficients admit a causal reading, and because the factorial is complete, the independence of the components can be tested. The components determine the process of the search rather than its outcome. The reporting style has the largest effect in the design and cuts the wasted trial ratio from 0.106 to 0.031. The stopping rule is the only component with an effect on either outcome metric, and its effect on the gain over default is smaller than the difference between the two datasets.

## 1 Introduction

### 1.1 Background

In an autoresearch loop, a language model performs the experimentation itself. It is given a goal, a body of code it may edit, and an executor that runs a trial and returns a scalar result. It then works unattended: it proposes a change, executes it, reads the outcome, decides what to try next, and repeats until the budget is exhausted. Karpathy’s autoresearch [5] fixed the current meaning of the term; it appeared in March 2026 and has since been forked several thousand times.

The architecture is a contract between three files. `prepare.py` holds the data and defines the score; nobody edits it, so every experiment is scored identically. `train.py` holds the model and the training loop and is edited only by the agent; `program.md` holds the instructions and is edited only by the human.

The experiment cycle exists only as English prose in `program.md`, and the model itself is the execution layer. `program.md` is therefore not configuration supplied to a controller; it *is* the controller.

### 1.2 Motivation and Research Objective

An instruction file settles five matters at once: what counts as success, how widely the agent may search, when it must stop, how it must report, and where to concentrate effort. Two differently worded files therefore differ in all five at once, so no effect can be attributed to any single matter. Separating them requires a file in which one matter changes while the rest stay fixed, which is to treat the file as a configuration vector rather than as prose. Five components of `program.md` are each written at two

levels, and all  $2^5 = 32$  combinations are generated and executed. The aim is not to propose a better autoresearch loop, but to establish what an existing one responds to.

Every comparison is between two configurations of the same system; no human baseline is used. The research questions are:

**RQ1** Which components of `program.md` causally determine which aspects of the loop’s research process, and how large is each effect?

**RQ2** Do the components act independently of one another, or does the effect of one depend on the level of another?

### 1.3 Structure of the Thesis

This thesis has four sections. Section 2 describes how the instruction file becomes a configuration vector, and why executing every configuration identifies the effect of each component. Section 3 gives the experimental setup, the metrics and the regression model, then the results. Section 4 summarizes the findings. Appendix A reproduces the three files the loop runs on, including the complete specification from which every instruction file can be reconstructed.

## 2 Background

Figure 1 shows the control flow of one run.

### 2.1 The Instruction File as a Configuration

Every instruction file is generated from a single template. The fixed part specifies the research question, the executor interface and the JSON response format. The variable part consists of five paragraphs, each written at one of two levels; these five slots are the independent variables. Nothing else in the file varies. A file is described completely by

$$c = (c_1, \dots, c_5), \quad c_i \in \{0, 1\},$$

where the slots are taken in the order metric, breadth, stopping rule, output format and emphasis, yielding  $2^5 = 32$  files. For example,  $c = (1, 0, 1, 1, 0)$  is the file `M1-B0-S1-O1-E0`: the level-1 paragraph in the evaluation slot, level 0 in breadth, level 1 in stopping, level 1 in reporting and level 0 in emphasis. Its neighbor `M1-B0-S1-O0-E0` is the same file with one paragraph changed and all else fixed. The remaining thirty files are the other combinations.

Table 1 lists the levels. A production `program.md` hardcodes the baseline to be exceeded, specifies how failures are handled, fixes a reporting discipline and imposes a parsimony rule [2]. The five components in the table are drawn from those categories. One of them matters more than it looks: the loop carries no state except its own transcript. Before proposing the next experiment it can read only what it wrote in the hypothesis and interpretation fields of earlier turns. The reporting style *O* therefore controls how much of its own reasoning the loop can later read.

The ten paragraphs were written by a language model other than the agent and are reproduced in Appendix B.2.

## 3 Experiment Setup and Results

All experiments were implemented in Python; the files the loop runs on are reproduced in the appendices.

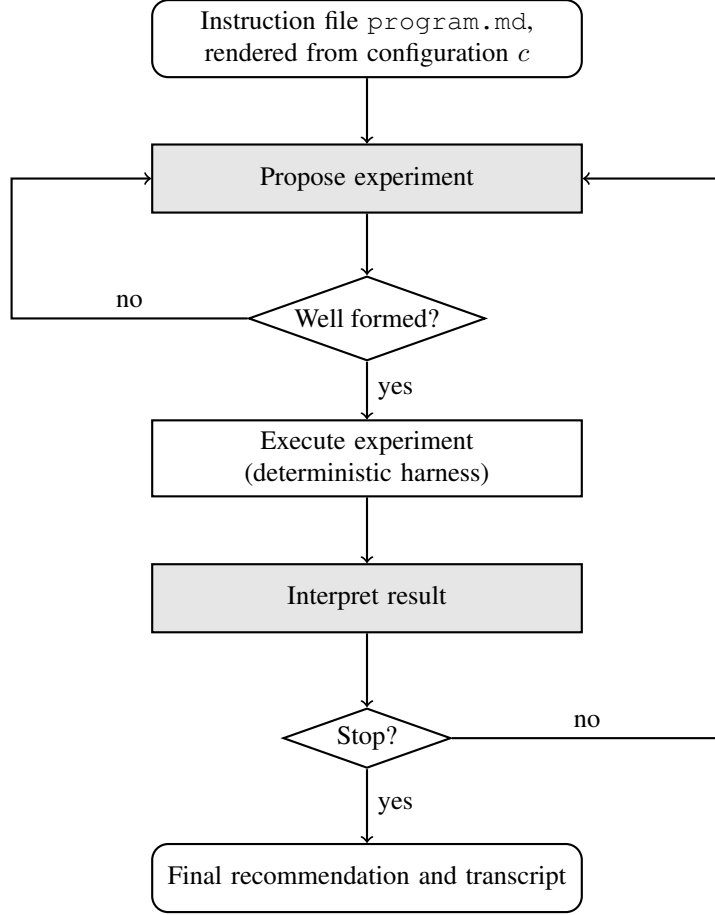


Figure 1: Control flow of one run. Shaded boxes are language-model calls. The unshaded box is deterministic, and hence every number the loop reasons about was measured and not generated. The validity check combines two checks in the code. The harness parses the reply and checks its shape, and `train.py` checks the whitelist before any estimator is fitted. A proposal that fails either check goes back to the proposal step. It is logged as an attempt, which is what the wasted trial ratio counts, and costs a turn but not an experiment. A run ends on the model’s stop decision, on eight executed experiments, or on four rejected or malformed replies, whichever comes first.

### 3.1 Tasks, Executor and Baseline

Every configuration runs on two canonical classification datasets, selected to differ in size, dimensionality and number of classes: Breast Cancer Wisconsin ( $569 \times 30$ , two classes) and Wine ( $178 \times 13$ , three classes). The loop may select `RandomForestClassifier` or `GradientBoostingClassifier` under a fixed hyperparameter whitelist that the agent is not shown, given in Appendix D. Anything outside it is rejected before an estimator is built.

A single reference quantity is computed per dataset before any run, without a language model. The quantity  $a_0(d)$  is the accuracy of a default-hyperparameter fit, taken as the better of the two model families, and every run is measured against it. On both datasets the better family is the random forest: on the cross-validated scale,  $a_0 = 0.9598$  on Breast Cancer and  $0.9600$  on Wine.

### 3.2 Design and Budget

The design is the complete  $2^5$  factorial crossed with the task suite and replicated  $R$  times:  $N = 2^5 \cdot D \cdot R$  runs. With  $D = 2$  and  $R = 20$  that is  $N = 1,280$  runs in 64 cells of 20. Of them, 1,278 ended on the model’s own stop decision and two on the cap of eight experiments. The study issued about 9,900 model

Table 1: The five varied components of `program.md`. A configuration is one choice of level per row. The last column gives the token count of each level’s paragraph, level 0 / level 1; the levels are not length-matched.

| Code     | Component            | Level 0                                                              | Level 1                                                                                                                 | Tokens  |
|----------|----------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|---------|
| <i>M</i> | Evaluation criterion | Success left undefined; the loop judges quality as it sees fit.      | Success defined as five-fold cross-validated accuracy, with the across-fold standard deviation used only to break ties. | 10 / 56 |
| <i>B</i> | Search breadth       | Commit to one model family for the session.                          | Compare both families within a session and vary several hyperparameters at once.                                        | 32 / 26 |
| <i>S</i> | Stopping rule        | Fixed budget of exactly three experiments.                           | Adaptive budget of up to eight, halted when an experiment fails to improve by more than 0.002.                          | 32 / 59 |
| <i>O</i> | Reporting style      | Terse: the action taken and the numbers involved.                    | Full sentences stating hypothesis, expectation and interpretation.                                                      | 20 / 34 |
| <i>E</i> | Search emphasis      | Explore first: several different configurations before refining any. | Exploit first: refine a promising configuration immediately.                                                            | 24 / 32 |

calls and 4,768 estimator fits, the fits taking 74 minutes of executor time in total.

### 3.3 Metrics

Four dependent variables are used.

**Outcome metrics.** The *gain over default* is the difference between the score of the model the run finally recommends and  $a_0(d)$ . It answers the primary question, whether the tuning achieved anything untuned defaults would not have given. A negative value means the run ended on a configuration worse than untuned defaults.

The *improvement rate* is the proportion of executed trials whose score is strictly higher than the best score earlier in the same run.

**Efficiency metrics.** The *wasted trial ratio* is the number of proposals rejected before execution, together with those duplicating an earlier proposal in the same run, divided by the proposals attempted.

The *cost to best* is the number of executor calls up to and including the one that produced the run’s highest score, taking the earliest such call on ties.

**Measurement window.** The stopping rule determines how many experiments a run performs, and every metric above aggregates over those experiments. All four are therefore computed over the first three experiments, the number performed at the fixed-budget level and hence available in every run.

### 3.4 Regression Model and Inference

Each component is coded as an indicator variable

$$X_i = \begin{cases} +1 & \text{if component } i \text{ is at level 1,} \\ -1 & \text{if at level 0,} \end{cases} \quad i = 1, \dots, 5, \quad (1)$$

Table 2: Upper panel: per-run quantities across all 1,280 runs. Lower panel: per-run means by level of the stopping rule  $S$  and the reporting style  $O$ . Study totals: 5,140 proposals attempted, 4,768 experiments executed, 372 proposals rejected, 5 duplicates.

| Per run                                                                     | Mean          | Median                 | Max |
|-----------------------------------------------------------------------------|---------------|------------------------|-----|
| Proposals attempted                                                         | 4.02          | 3                      | 9   |
| Experiments executed                                                        | 3.73          | 3                      | 8   |
| Proposals rejected                                                          | 0.29          | 0                      | 2   |
| Duplicate proposals                                                         | 0.004         | 0                      | 1   |
| Executor calls to best                                                      | 1.71          | 1                      | 7   |
| <i>(complete runs; within the three-experiment window the maximum is 3)</i> |               |                        |     |
| Per run (mean)                                                              | $S = 0$ fixed | $S = 1$ adaptive       |     |
| Experiments executed                                                        | 2.88          | 4.57                   |     |
| Proposals rejected                                                          | 0.17          | 0.41                   |     |
| Experiments after best                                                      | 1.30          | 2.73                   |     |
| Per run (mean)                                                              | $O = 0$ terse | $O = 1$ full sentences |     |
| Experiments executed                                                        | 3.76          | 3.69                   |     |
| Proposals rejected                                                          | 0.43          | 0.15                   |     |

and the outcome of one run is modeled as

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{j=2}^D \gamma_j D_j + \varepsilon, \quad (2)$$

where  $D_j$  indicates that the run was made on dataset  $j$ , with the first dataset as reference. The  $\gamma_j$  are dataset fixed effects. They absorb the inherent difficulty of each task, its headroom above  $a_0(d)$  and its response to the whitelist. Here  $\beta_0$  is the mean across configurations on the reference dataset,  $\beta_i$  the effect of component  $i$ , and  $\varepsilon$  run-to-run error.  $2\beta_i$  is the average change in  $Y$  produced by switching component  $i$ , and  $2\hat{\beta}_i$  is reported throughout. Because all thirty-two configurations are executed, the interaction model is also identified,

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{1 \leq i < j \leq 5} \beta_{ij} X_i X_j + \sum_{j=2}^D \gamma_j D_j + \varepsilon, \quad (3)$$

and this regression tests RQ2.

Both models are fitted by ordinary least squares on the individual runs, with heteroskedasticity-consistent (HC3) standard errors and Benjamini–Hochberg adjustment within each metric.

### 3.5 Results

**What the runs did.** Table 2 describes the 1,280 runs. The loop attempted 5,140 proposals, of which 4,768 were executed and 372 rejected; only five exactly duplicated an earlier one. Not one of the 5,140 replies was malformed JSON or of the wrong shape: 369 of the 372 rejections were whitelist violations, 365 of them on the random forest and four on gradient boosting, and the remaining three were failed estimator fits. The hyperparameters the loop reached for were legitimate scikit-learn arguments that the whitelist does not admit, above all `min_samples_split` on the random forest, which accounts for 290 of the 369 violations, followed by `random_state` with 71. Here the wasted trial ratio is therefore almost entirely the rate at which the loop names a hyperparameter outside an interface it has not been shown.

The best score arrived on the first executor call in 720 runs (56.2%), on the second in 291 (22.7%), on the third in 222 (17.3%), and on a later call in only 47 runs (3.7%); no run needed more than seven calls.

Table 3: Main-effect estimates  $2\hat{\beta}_i$  per metric from equation (2), with heteroskedasticity-consistent standard errors and Benjamini–Hochberg adjusted  $p$ -values. The  $\hat{\gamma}$  column is the fitted dataset fixed effect (Wine relative to Breast Cancer). Stars mark adjusted  $p < 0.05$  (\*),  $< 0.01$  (\*\*) and  $< 0.001$  (\*\*\*).

| Metric             | $M$       | $B$       | $S$       | $O$       | $E$    | $\hat{\gamma}$ | $R^2$ |
|--------------------|-----------|-----------|-----------|-----------|--------|----------------|-------|
| Gain over default  | +0.001    | -0.001    | +0.001**  | +0.000    | +0.000 | -0.001***      | 0.029 |
| Wasted trial ratio | -0.023*** | -0.034*** | +0.044*** | -0.075*** | +0.005 | -0.003         | 0.159 |
| Improvement rate   | +0.008    | -0.026*   | +0.034**  | +0.006    | +0.018 | -0.064***      | 0.044 |
| Cost to best       | +0.06     | -0.02     | +0.05     | -0.01     | -0.11  | -0.20***       | 0.025 |

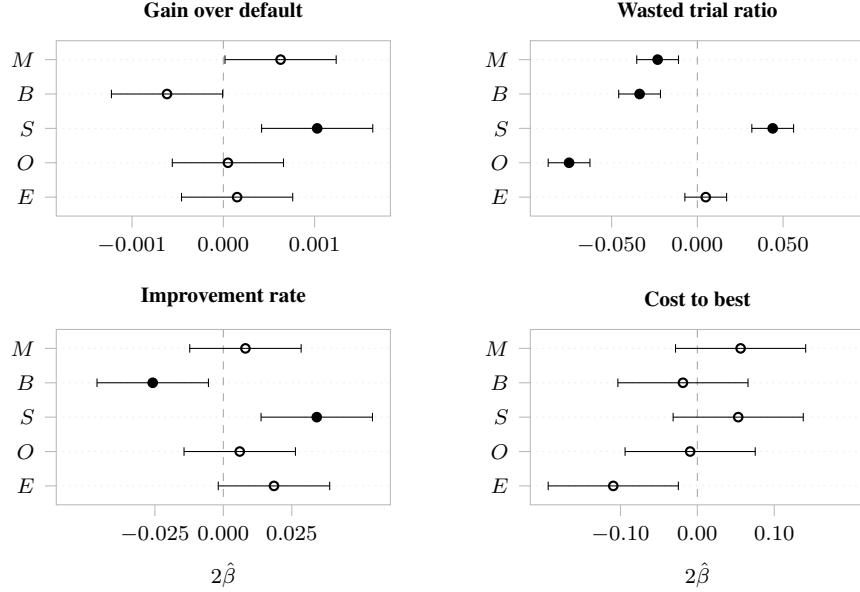


Figure 2: Estimated component effects on every metric. Points are  $2\hat{\beta}_i$  from equation (2), bars are 95% intervals built from heteroskedasticity-consistent standard errors, and a filled point marks an effect that survives Benjamini–Hochberg adjustment at 0.05. The dataset fixed effect is in the model, so these are within-task effects. The four panels carry independent horizontal scales, the metrics being in different units.

In all, 96.3% of runs never improved after their third trial. On average 2.02 experiments per run ran after the best had already been found: 2,585 of the 4,768 executed trials, more than half the compute in the study. On the gain over default, 40.0% of runs improved on  $a_0$ , 39.1% recommended a configuration scoring exactly  $a_0$ , and 20.9% ended below it.

**Main effects.** Table 3 gives the main-effect estimates and Figure 2 plots them with their confidence intervals.

The wasted trial ratio is the most responsive metric. Four of the five components produce effects that survive adjustment, and the model accounts for  $R^2 = 0.16$  of its variance. The reporting style produces the largest effect anywhere in the design,  $2\hat{\beta}_O = -0.075$ : the mean ratio falls from 0.106 at level 0 to 0.031 at level 1. Given the composition of the rejections, this is a reduction in whitelist violations: 273 of the 369 occurred in runs with terse reporting and 96 in runs with full sentences. Search breadth and the evaluation criterion act in the same direction, with effects of  $-0.034$  and  $-0.023$ . The adaptive stopping rule acts in the opposite direction,  $+0.044$ , raising the mean ratio from 0.046 to 0.090. The mechanism is visible in the descriptives. The adaptive level raises the mean number of executed experiments from 2.88 to 4.57 and the mean number of rejected proposals per run from 0.17 to 0.41. The additional proposals do not duplicate earlier ones, of which there are only five in the study, but reach more often for

a hyperparameter outside the whitelist: 260 of the 369 violations occur under the adaptive rule.

The two outcome metrics, by contrast, respond to the stopping rule alone. It is the only component whose effect on the gain over default survives adjustment (+0.001, adj.  $p = 0.005$ ), and it raises the improvement rate from 0.135 to 0.170. Search breadth is the only other component affecting the improvement rate, at  $-0.026$ . No component affects the cost to best at the adjusted 0.05 level; the largest estimate is exploit-first at  $-0.11$ , with adj.  $p = 0.057$ . The components that determine efficiency are therefore not the components that determine outcome.

On two of the four metrics the dataset fixed effect is roughly twice the largest component effect. Wine reduces the improvement rate by 0.064 and the cost to best by 0.20 relative to Breast Cancer, both at  $p < 10^{-5}$ , against largest component effects of 0.034 and 0.11 on the same metrics. On these two metrics the task matters more than the wording of the instruction file, and this is exactly the variation equation (2) removes from the residual.

**Interactions.** Table 4 gives the two-way interactions. Three of the forty estimated across the four metrics survive adjustment; for the remaining thirty-seven the data provide no evidence of interaction. All three occur on the wasted trial ratio and all three involve the reporting style:  $S \times O$  at  $-0.031$ ,  $B \times O$  at  $+0.018$ , and  $O \times E$  at  $-0.016$ . Of these, only  $S \times O$  is comparable in magnitude to a main effect, and its sign indicates complementarity rather than substitution. The cell means show the direction: the adaptive budget raises the mean wasted trial ratio from 0.068 to 0.143 when reporting is terse, but only from 0.024 to 0.037 when it is in full sentences, so that the additive prediction for the adaptive-and-verbose cell, 0.100, overstates the observed 0.037 by a factor of nearly three. Equivalently, the reduction the reporting style buys is larger under an adaptive budget ( $-0.106$ ) than under a fixed one ( $-0.043$ ). A generous experiment budget is expensive in wasted proposals only when the loop is also reporting tersely. The remaining two interactions are smaller and of opposite character:  $B \times O$  is the substitution case, both components reducing waste and each reducing it less when the other is already at level 1, while  $O \times E$  resembles  $S \times O$ . Adding all ten interaction terms raises  $R^2$  on that metric only from 0.159 to 0.192. The same three terms survive adjustment, with unchanged signs, when the ratio is refitted on the logit, square-root or arcsine-square-root scale, so the interaction structure is not an artifact of fitting a proportion on its natural scale. The components may therefore be treated as additive, which is the answer to RQ2, and the complete factorial is what allows the question to be tested rather than assumed.

**Compliance.** Table 4 also reports the share of runs whose behavior matched the instruction, so a component with no effect can be told apart from one the model ignored.  $B$  is checked by the number of model families a run proposes,  $S$  by the number of experiments it performs and  $E$  by whether its second proposal refines its first;  $O$  constrains wording rather than action, so it is checked by the mean characters of model-written text per step;  $M$  leaves no behavioral trace, because the executor computes the score whatever the instruction says. The fixed budget is followed in 89% of runs and the adaptive rule in only 20%, so the stopping-rule estimates measure the effect of offering an adaptive budget rather than of a loop that reliably stops on diminishing returns. Every estimate in Table 3 is intention-to-treat.

## 4 Conclusion

In an autoresearch loop, `program.md` does what controller code does in conventional software, yet cannot be read like code. This thesis has treated it as a configuration vector. Five components were written at two levels each, all  $2^5 = 32$  configurations were generated and executed against a real executor on two datasets with twenty replicates each, and the behavior of the 1,280 runs was reduced to four metrics and regressed on an orthogonal  $\pm 1$  coding of the components. Because the configurations were constructed rather than collected, the coefficients admit a causal reading. Because the factorial is complete, the independence of the components could be tested rather than assumed.

Table 4: Two-way interaction estimates  $2\hat{\beta}_{ij}$  from equation (3), and compliance per component. The four interactions with the smallest adjusted  $p$  are shown, with adjusted  $p$  given directly; compliance is reported as level 0 / level 1.

| Interaction                       | Estimate | adj. $p$ | Component | Compliance      |
|-----------------------------------|----------|----------|-----------|-----------------|
| $S \times O$ (wasted trial ratio) | -0.031   | 0.000    | $B$       | 0.66 / 0.82     |
| $B \times O$ (wasted trial ratio) | +0.018   | 0.009    | $S$       | 0.89 / 0.20     |
| $O \times E$ (wasted trial ratio) | -0.016   | 0.019    | $E$       | 0.88 / 0.69     |
| $M \times B$ (wasted trial ratio) | -0.010   | 0.171    | $O$       | 205 / 386 chars |
|                                   |          |          | $M$       | not observable  |

## 4.1 Contributions

The contributions are as follows.

1. **A method for attributing the behavior of an autoresearch loop to its instruction file.** The instruction file is partitioned into slots, each slot is written at two levels by a model distinct from the agent, the complete factorial is executed, and the effects are estimated by a regression whose orthogonal coding and dataset fixed effect follow from the design. The method is not tied to the five components varied here.
2. **An answer to RQ1.** The components determine the process, not the outcome. The reporting style produced the largest effect in the design, cutting the mean wasted trial ratio from 0.106 to 0.031, and search breadth and the evaluation criterion moved the same metric by  $-0.034$  and  $-0.023$ . Almost every wasted trial was a proposal naming a hyperparameter outside an undisclosed whitelist, and none was a malformed reply. The stopping rule was the only component with an effect on either outcome metric,  $+0.001$  on the gain over default and  $+0.034$  on the improvement rate, and no component determined the cost to best. On two of the four metrics the dataset fixed effect exceeded every component effect, which is the reason equation (2) controls for it.
3. **An answer to RQ2.** Thirty-seven of the forty two-way terms show no evidence of interaction. The three that survive adjustment are all on the wasted trial ratio and all involve the reporting style, and together the ten interaction terms raise  $R^2$  on that metric only from 0.159 to 0.192, so the components may be treated as additive.

With untuned defaults already at  $a_0 \approx 0.96$ , the instruction file determines how the loop conducts its search rather than what the search returns.

## References

- [1] T. Lütke. Performance: 53% faster parse+render, 61% fewer allocations. Pull request #2056, Shopify/liquid, 2026. Produced by an autoresearch run of approximately 120 experiments over 93 commits; unmerged at the time of writing. <https://github.com/Shopify/liquid/pull/2056>.
- [2] B. Tychiev. A guide to Andrej Karpathy’s AutoResearch: automating ML with AI agents. DataCamp tutorial, 23 March 2026. Secondary source; it reports the contents of the repository’s program.md and results log. <https://www.datacamp.com/tutorial/guide-to-autoresearch>.
- [3] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. Nature, 624(7992):570–578, 2023. doi:10.1038/s41586-023-06792-0.

- [4] Q. Huang, J. Vora, P. Liang, and J. Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. In Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR 235:20271–20309, 2024. arXiv:2310.03302.
- [5] A. Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. <https://github.com/karpathy/autoresearch>.
- [6] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. arXiv:2408.06292.
- [7] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models’ sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. ICLR, 2024. arXiv:2310.11324.
- [8] A. Webson and E. Pavlick. Do prompt-based models really understand the meaning of their prompts? NAACL-HLT, pages 2300–2344, 2022.

## A The files the loop runs on

Section 1 describes the three-file contract of Karpathy’s autoresearch: `prepare.py` fixes the data and the score, `train.py` holds the model and the training loop, and `program.md` holds the instructions. The study keeps that contract file for file. All three are reproduced below: the specification of `program.md` in Appendix B, `prepare.py` in Appendix C, and `train.py` together with the executor whitelist in Appendix D.

One thing does differ, and it is the narrowing of Section 2 rather than a departure from the contract. In Karpathy’s loop the agent edits `train.py` directly. Here the search space is a hyperparameter space, not arbitrary code, so that the instruction file stays the only free variable; the agent names a model family and hyperparameter values in a JSON proposal and `train.py` builds exactly that estimator. The agent therefore acts on `train.py` through a fixed interface instead of rewriting it. What is unchanged is the ownership that matters for measurement: `prepare.py` is reachable by nobody, so no level of any component can alter what a score means, and every number in a transcript is a measurement rather than a claim.

## B Specification of the instruction file

A `program.md` is not written; it is rendered. This appendix gives the whole specification, from which any of the thirty-two files can be reconstructed exactly: the template, which is fixed, and the ten paragraphs that fill its five slots.

Appendix B.1 gives the template. Everything in it is byte-identical across all thirty-two configurations of a task. The five marked slots are the independent variables. The three remaining substitutions, `task_title`, `research_question` and `dataset_name`, are set by the task and not by the configuration, so they are constant across the thirty-two files of one dataset and differ only between datasets.

Appendix B.2 gives the ten paragraphs, two per slot, verbatim. A configuration  $c = (c_1, \dots, c_5)$  selects one paragraph per slot, and substituting one paragraph for the other level of the same slot is the only difference between two files. These ten texts are the treatment: they were written once by a language model from fixed meta-prompts that were not preserved, then frozen and reused word for word, for the reason given in Section 2. Table 1 summarizes them; what follows is the text itself, since it is the wording rather than the summary that the loop responds to.

### B.1 Template

```

# Research Task: {{ task_title }}

You are an autonomous research agent. Your task is to investigate the following
question through real, executed experiments (not simulated ones):

> {{ research_question }}

You have access to a fixed dataset ({{ dataset_name }}), already loaded and split
into train/validation) and to a tool that fits either a RandomForestClassifier or
a GradientBoostingClassifier with hyperparameters you choose. The tool always
returns the same record: 5-fold cross-validated accuracy on the training split,
its standard deviation across folds, and accuracy on the held-out validation
split.

## Evaluation

{ slot M --- metric }
{{ metric_text }}

## Search strategy

{ slot B --- breadth }
{{ breadth_text }}

{ slot E --- emphasis }
{{ emphasis_text }}

## Budget and stopping

{ slot S --- stopping }
{{ stopping_text }}

## Reporting style

{ slot O --- output format }
{{ output_format_text }}

## Required response format

At every step, respond with a single JSON object and nothing else, matching one
of these two shapes:

Proposing an experiment:
'''json
{"action": "propose", "hypothesis": "<why you think this configuration will help>",
 "model": "random_forest" | "gradient_boosting",
 "params": {"<hyperparameter>": <value>, ...}}
'''

Interpreting the last result and deciding what happens next:
'''json
{"action": "interpret", "interpretation": "<what the result tells you>",
 "decision": "continue" | "stop",
 "final_recommendation": "<only required when decision == stop: your final
 recommended model and hyperparameters, and why>"}
'''

```

## B.2 The ten treatment paragraphs

**slot M --- Evaluation criterion** (substituted for {{ metric\_text }})  
**level 0:** Evaluate model quality in whatever way you judge appropriate.

**level 1:** Evaluate every model using 5-fold cross-validated accuracy on the training split, AND report the standard deviation across folds as a measure of stability. A configuration is only better than another if its mean accuracy is higher; use the standard deviation only to break ties or to flag instability.

**slot B --- Search breadth** (substituted for {{ breadth\_text }})  
**level 0:** Stay within a single model family per experiment. Pick either RandomForestClassifier or GradientBoostingClassifier for the whole session unless you have exhausted reasonable hyperparameters for it and explicitly decide to switch.

**level 1:** You are free to compare RandomForestClassifier and GradientBoostingClassifier side by side within the same session, and to vary multiple hyperparameters of each at once.

**slot S --- Stopping rule** (substituted for {{ stopping\_text }})  
**level 0:** You have a fixed budget of exactly 3 experiments in total. Run exactly 3, then stop and report your final recommendation, regardless of the outcome of experiment 3.

*level 1:* You have a budget of up to 8 experiments. After each experiment, decide whether to continue: stop as soon as an experiment fails to improve the evaluation metric (defined above) by more than 0.002 over the best result so far, or when you reach 8 experiments, whichever comes first.

**slot O --- Reporting style** (substituted for {{ output.format.text }})

*level 0:* Keep every response short: state the action you are taking and the numbers involved, with minimal prose.

*level*

*1:* Explain your reasoning at every step in full sentences: what you hypothesize, why, what you expect to observe, and how you interpret the actual result relative to that expectation.

**slot E --- Search emphasis** (substituted for {{ emphasis.text }})

*level 0:* Prioritize breadth of search: try several meaningfully different configurations before you spend more than one experiment refining any single one of them.

*level 1:* Prioritize depth: as soon as one configuration looks promising, immediately refine it with nearby variations (e.g. slightly different hyperparameter values) before trying anything unrelated.

## C prepare.py

The data and the score. Nobody edits this file, and the agent never sees it, so no level of any component can change what an experiment is measured on or how it is scored. One stratified 70/30 partition per task, fixed with the same seed for every run, and one scoring procedure returning the same record every time, whatever a configuration's evaluation paragraph emphasizes.

"""

prepare.py -- the data and the score. Nobody edits this file.

The first of the three files of the autoresearch contract (thesis, Introduction). It fixes what every experiment is measured on and how it is scored, so that a score is comparable across configurations, across replicates and across the whole study. Neither the agent nor the instruction file can reach it: the agent never sees this module, and no level of any component in program.md changes what it returns. That is what makes a number in a transcript a measurement rather than a claim.

Two responsibilities, and no others:

```
load(dataset)           the fixed train/validation partition for one task
score(estimator, dataset) the record every experiment is scored by
```

Every task uses the identical split protocol: one stratified 70/30 partition fixed with the same seed across all runs, so the split is a constant of the study rather than a source of variation (thesis, Experimental setup). The score is the same record whatever a configuration's evaluation paragraph emphasises: cross-validated accuracy on the training split, its across-fold standard deviation, and accuracy on the held-out validation split.

Adding a dataset here does not put it in the study; the suite is whatever study\_config.DATASETS says.

"""

```
import sys
from functools import lru_cache
from pathlib import Path
```

```
from sklearn import datasets as skds
from sklearn.model_selection import cross_val_score, train_test_split
```

```
sys.path.insert(0, str(Path(__file__).resolve().parent / "config"))
from study_config import CV_FOLDS, SCORING, SPLIT_SEED, TEST_SIZE # noqa: E402
```

# name -> zero-argument loader returning (X, y)

```
LOADERS = {
    "breast_cancer": lambda d: (d.data, d.target)(skds.load_breast_cancer()),
    "wine":          lambda d: (d.data, d.target)(skds.load_wine()),
    "digits":        lambda d: (d.data, d.target)(skds.load_digits()),
    "iris":           lambda d: (d.data, d.target)(skds.load_iris()),
}
```

```
@lru_cache(maxsize=None)
```

```
def load(name):
    """Return the fixed split for one dataset as (X_train, X_val, y_train, y_val)."""
    if name not in LOADERS:
```

```

        raise KeyError(f"unknown dataset {name!r}; known: {sorted(LOADERS)}")
X, y = LOADERS[name]()
return train_test_split(
    X, y, test_size=TEST_SIZE, random_state=SPLIT_SEED, stratify=y
)

def score(estimator, dataset):
    """The one scoring procedure of the study. Returns the record, or raises."""
    X_tr, X_va, y_tr, y_va = load(dataset)
    cv = cross_val_score(estimator, X_tr, y_tr, cv=CV_FOLDS, scoring=SCORING)
    estimator.fit(X_tr, y_tr)
    val = estimator.score(X_va, y_va)
    return {
        "cv_accuracy_mean": round(float(cv.mean()), 6),
        "cv_accuracy_std": round(float(cv.std()), 6),
        "val_accuracy": round(float(val), 6),
    }

def describe(name):
    import numpy as np
    X_tr, X_va, y_tr, y_va = load(name)
    return {
        "dataset": name,
        "n_train": int(X_tr.shape[0]),
        "n_val": int(X_va.shape[0]),
        "n_features": int(X_tr.shape[1]),
        "n_classes": int(len(np.unique(y_tr))),
    }

```

## D train.py and the executor whitelist

The model and the training loop. The whitelist here is the boundary of the search space: a proposal naming an unknown family, or a hyperparameter outside the whitelist for its family, is rejected before it reaches the scoring procedure, so a malformed proposal costs a turn but not an experiment. The whitelist is defined once in the study configuration and is the same on every task; it is not shown to the agent, whose instruction file only says that the hyperparameters are its to choose.

```

# config/study_config.py (excerpt): the executor whitelist
ALLOWED_PARAMS = {
    "random_forest": {"n_estimators", "max_depth", "min_samples_leaf", "max_features"},
    "gradient_boosting": {"n_estimators", "max_depth", "learning_rate", "subsample"},
}

"""
train.py -- the model and the training loop.

```

The second file of the autoresearch contract (thesis, Introduction). In Karpathy's loop this is the file the agent edits, and it is the only one it may edit. The loop studied here narrows the search space from arbitrary code to a hyperparameter space, so that the instruction file stays the only free variable (thesis, The loop under study). The narrowing changes how the agent reaches this file, not which file it acts on: instead of rewriting the source, the agent names a model family and hyperparameter values in a JSON proposal, and this module builds and trains exactly that estimator. Everything the agent can vary about the model lives here; nothing it can vary about the measurement does, that being prepare.py.

The whitelist is the boundary of the search space. A proposal naming an unknown family, or a hyperparameter outside the whitelist for its family, is rejected here and never reaches the scoring procedure, which is what keeps a malformed proposal from costing an experiment.

```

    run_experiment(model, params, seed, dataset) -> record | {"error": ...}
"""
import sys
import time
from pathlib import Path

from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier

STUDY = Path(__file__).resolve().parent
sys.path.insert(0, str(STUDY))
sys.path.insert(0, str(STUDY / "config"))
import prepare # noqa: E402

```

```

import study_config # noqa: E402 (read live, so WORKERS can set EXECUTOR_N_JOBS)
from study_config import ALLOWED_PARAMS # noqa: E402

def build(model, params, seed):
    """The estimator named by a proposal. Raises on an unknown family."""
    if model == "random_forest":
        # n_jobs is set to 1 when the outer loop is parallelised, so that the
        # forest's internal threads do not oversubscribe the cores.
        return RandomForestClassifier(
            random_state=seed, n_jobs=study_config.EXECUTOR_N_JOBS, **params)
    if model == "gradient_boosting":
        return GradientBoostingClassifier(random_state=seed, **params)
    raise ValueError(f"unknown model {model!r}")

def validate(model, params):
    """Whitelist check. Returns an error string, or None if the proposal is legal."""
    if model not in ALLOWED_PARAMS:
        return f"unknown model '{model}', must be one of {sorted(ALLOWED_PARAMS)}"
    bad = set(params) - ALLOWED_PARAMS[model]
    if bad:
        return f"unsupported params for {model}: {sorted(bad)}"
    return None

def run_experiment(model, params, seed, dataset):
    """Train one estimator on one dataset and score it. Record, or {'error': ...}."""
    err = validate(model, params)
    if err:
        return {"error": err}

    t0 = time.time()
    try:
        record = prepare.score(build(model, params, seed), dataset)
    except Exception as exc:
        # noqa: BLE001 - reported to the model
        return {"error": f"experiment failed: {exc}"}

    return {"model": model, "params": params, "dataset": dataset, **record,
            "wall_time_s": round(time.time() - t0, 3)}

```